

Islamic University of Technology

Department of Computer Science and Engineering (CSE)
B.Sc. in Software Engineering

SWE 4404: Software Project Lab II
Academic Year: 2024–2025

Software Requirements Specification (SRS)

Plastic Box Solver

An Interactive Browser-Based 3D Rubik's Cube Computer Vision Scanner, Solver, and Coaching Suite

Group 6

S. M. Tahsinuzzaman Emon (ID: 230042104) smtemon@iut-dhaka.edu
S. M. Samiul Hossain (ID: 230042110) samiulhoassain@iut-dhaka.edu
Md. Muntahi Hasan Akhiar (ID: 230042118) muntahihasan@iut-dhaka.edu

Supervisor

Aashnan Rahman
Junior Lecturer, Department of Computer Science and Engineering (CSE)
Islamic University of Technology (IUT)

Date: October 2026

Document Revision History

This Software Requirements Specification (SRS) is a living engineering document that originated during the initial requirement collection phase of SPL-II and iteratively evolved across the System Design and Implementation milestones.

Version	Date	Primary Author(s)	Summary of Changes & Evolution Rationale
v1.0	May 08, 2026	Emon, Samiul, Akhiar	Initial requirements collection and baseline SRS for Project Proposal. Established FR-01 through FR-22 based on user interviews and tool audits.
v1.1	July 27, 2026	Samiul, Emon, Akhiar	Design session revision. Formalized 3-layer architecture and data models. Added **FR-23** (Cross-Device Saved Cube Slots) per suggestion by industry panelist at IUT. Detailed beginner vs. optimal solving models.
v1.2	Oct 29, 2026	Emon, Samiul, Akhiar	Final consolidated SRS for SWE 4404 submission. Added **FR-24** (Interactive Stage Tutorials), 7-stage CFOP live coaching breakdown, and verified final implementation statuses. Documented deferred scope items (FR-15, FR-19, FR-21).

Table 1: Document Revision and Engineering History

Source Repository: `https://github.com/SMTEmon/plastic-box-solver.git`
Requirement Collection Spreadsheet (Live): Google Sheets Requirement Collection Link

Contents

Document Revision History . . . . . **i**

1 Introduction **1**
1.1 Purpose 1
1.2 Document Conventions and Selected Template 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Project Scope and Product Value Proposition 2
1.5 Requirement Elicitation Methodology 2
1.6 References 2

2 Overall Description **3**
2.1 Product Perspective and System Boundary 3
2.2 Literature Review and Existing Solutions Gap Analysis 3
2.3 User Classes and Stakeholder Characteristics 4
2.4 Operating Environment 4
2.5 Design and Implementation Constraints 4
2.6 Assumptions and Dependencies 4

3 System Features and Functional Requirements **5**
3.1 Feature Group 1: User Account and Profile Management 5
3.2 Feature Group 2: Cube Input, Computer Vision, and Validation 7
3.3 Feature Group 3: Interactive 3D Visualization and Manipulation 9
3.4 Feature Group 4: Solving Algorithms and Educational Tutorials 11
3.5 Feature Group 5: Solve Execution Modes and Live Coaching 13
3.6 Feature Group 6: Telemetry, Persistence, Saved Cubes, and Leaderboard 14

4 External Interface Requirements **17**
4.1 User Interfaces 17
4.2 Hardware Interfaces 18
4.3 Software Interfaces 18
4.4 Communications Interfaces 18

5 Other Non-Functional Requirements **19**

6 Traceability, Sustainability, and Project Feasibility **20**
6.1 Requirements Traceability Matrix 20
6.2 Contextual, Privacy, and Sustainable-Development Considerations 21
6.3 Effort, Infrastructure, and Cost Feasibility Estimates 21

A Requirement Collection Document and Live Link **22**

B Technical Glossary and Acronyms **29**

1 Introduction

1.1 Purpose

This document specifies the software requirements for **Plastic Box Solver**, an open-source, browser-based Rubik's Cube scanning, solving, and interactive coaching assistant. The system allows users to capture the physical state of a standard $3 \times 3 \times 3$ Rubik's Cube via camera or manual 2D net input, validates state legality, renders the puzzle as an interactive 3D model, and computes optimal or pedagogical step-by-step solutions (via Kociemba two-phase and CFOP algorithms). It also tracks solve metrics, persists personal history, and provides a global speedcubing leaderboard.

This specification serves as the formal functional baseline for developers, academic supervisors, quality assurance testers, and project evaluators throughout the software lifecycle of SWE 4404: Software Project Lab II.

1.2 Document Conventions and Selected Template

This SRS follows the widely recognized **Karl Wiegers Feature-Driven SRS Template**, which adapts and streamlines the formal standards of **IEEE Std 830-1998** and **ISO/IEC/IEEE 29148:2018**. The template emphasizes clear user-centric feature decomposition, explicit traceability from stakeholder needs, and normative requirements language.

Throughout this specification:

- The keyword **"shall"** denotes a mandatory, testable system requirement.
- The keyword **"should"** denotes a recommended capability or planned enhancement.
- Every functional requirement is tagged with an identification code (**FR-XX**), a priority rating (**High**, **Medium**, or **Low**), its final implementation status (**Implemented** or **Deferred**), the elicitation source (**E-01** through **E-05**), and the primary stakeholder persona (**SH-01** through **SH-04**).
- Every requirement block includes an explicit **Engineering Justification** documenting the rationale behind the design and implementation decisions.

1.3 Intended Audience and Reading Suggestions

This document is structured for the following primary stakeholder and technical groups:

- **Course Instructors and Academic Supervisors:** Should review Section 2 for project context, Section 3 for feature definitions, and Section 6 for Course Outcome (CO2/CO7) defense.
- **Software Developers and Maintainers:** Should focus on Section 3 (Functional Requirements), Section 4 (API and data DTO interfaces), and Section 5 (Quality attributes).
- **External Industry Evaluators:** Should examine Section 2.2 (Literature and Existing Solutions Gap Analysis), Section 3 (Feature breakdown), and Appendix A (Raw Requirement Collection Documentation).

1.4 Project Scope and Product Value Proposition

Millions of people acquire a Rubik's Cube each year, yet over 90% are unable to solve it unassisted. Existing online solutions suffer from severe usability friction: text-based algorithmic notation requires memorizing abstract sequences, static video tutorials require constant, disruptive context switching between physical hands and a video player, and commercial Bluetooth "smart cubes" cost upwards of $50 to $100, posing a steep economic barrier.

Plastic Box Solver eliminates these friction points by providing a zero-cost, zero-install, browser-accessible digital coaching assistant. Users can scan any standard, physical plastic cube using their device camera or 2D net editor, receive instant mathematical legality validation, and follow a synchronized 3D animated model that visually demonstrates precisely which layer to rotate. Furthermore, speedcubers can practice with an unassisted live timer, review animated 3D replays of their solves, and benchmark their move counts against optimal CFOP baselines on a global leaderboard.

1.5 Requirement Elicitation Methodology

The requirements in this specification were gathered and refined through five systematic elicitation techniques across the SPL-II development cycle:

E-1: Semi-Structured Beginner Interviews: Conducted with IUT undergraduate students who owned an unsolved Rubik's Cube. Identified physical-to-screen attention switching and notation ambiguity as the primary obstacles to learning.

E-2: Speedcuber User Observations: Evaluated competitive cubers practicing on campus. Identified the critical need for keyboard shortcuts (U, D, L, R, F, B), smooth animation without move-dropping, and fair efficiency scoring.

E-3: Literature Review and Software Audit: Evaluated five market benchmarks (*Cube Explorer, Grubiks, csTimer, JPerm Tutorials, and GAN Smart Cubes*) to isolate functional gaps.

E-4: Proposal Panel Feedback: Evaluators suggested avoiding trivial library wrappers by integrating structured, multi-stage teaching algorithms (leading to CFOP and interactive tutorials).

E-5: Design Presentation Industry Feedback: An industry guest evaluator at IUT recommended adding cross-device persistence for scanned cubes so users could scan on a mobile camera and solve later on a desktop screen.

1.6 References

1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
2. ISO/IEC/IEEE 29148:2018, *Systems and software engineering — Life cycle processes — Requirements engineering*.
3. Karl E. Wiegers and Joy Beatty, *Software Requirements*, 3rd ed., Microsoft Press, 2013.
4. Herbert Kociemba, *The Two-Phase Algorithm and Symmetry Coordinates for the Rubik's Cube*, 1992.
5. Jessica Fridrich, *CFOP (Cross, F2L, OLL, PLL) Speedcubing System*, 1997.

2 Overall Description

2.1 Product Perspective and System Boundary

Plastic Box Solver operates as a self-contained, three-tier web application consisting of a single-page frontend (React, Vite, Three.js), a high-performance RESTful API backend (FastAPI, OpenCV, Python), and an ACID-compliant relational database (PostgreSQL 16). The complete environment is containerized via Docker Compose.

Figure 1 illustrates the Level-0 Context Diagram, defining the system boundary and primary external data flows.

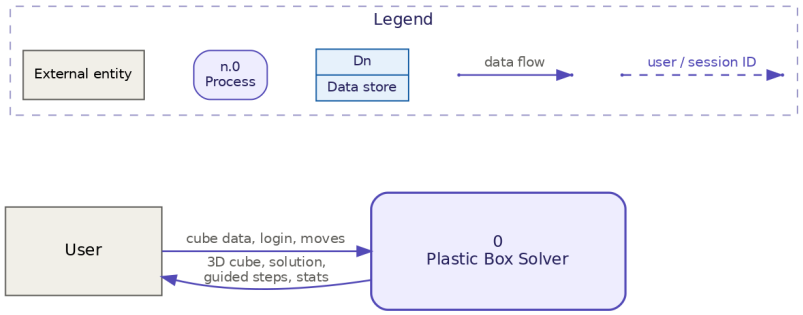


Figure 1: Level-0 Context Data Flow Diagram (DFD) defining system boundaries

2.2 Literature Review and Existing Solutions Gap Analysis

To establish the technical rationale for Plastic Box Solver, the team conducted a comparative literature review of existing solutions across the physical, digital, and academic domains (Table 2).

Existing Solution	Input Modality	Solving Method	Key Advantages	Identified Gaps & Limitations
Video Guides (e.g., JPerm, YouTube)	Manual visual matching	Beginner / CFOP methods	Rich visual commentary and finger-trick advice	Static; requires constant context-switching; cannot detect specific user mistakes.
Cube Explorer (Herbert Kociemba)	Manual net clicking	Kociemba Two-Phase (optimal ≤ 20 moves)	Extremely fast; mathematically optimal	Desktop Windows-only binary; no camera scanning; unusable for human learning.
Grubiks Web Solver	Manual 2D palette click	Layer-by-layer beginner	Web browser accessible; 3D step viewer	No camera scanning; ad-heavy; clunky mobile touch controls; lacks practice modes.
csTimer Speed Suite	Manual keyboard timer	Scramble generators only	Professional timing; extensive statistics	No 3D cube model; cannot solve or scan scrambled physical cubes.
Smart Bluetooth Cubes	Embedded gyros & hall sensors	Proprietary app CFOP / Optimal	Real-time hardware tracking; instant sync	Expensive ($40–$100+); proprietary hardware locks; creates electronic waste.
Plastic Box Solver (Proposed System)	**OpenCV camera scan + 2D Net**	**Optimal Kociemba + CFOP + Stage Coach**	**Zero install; camera auto-calibration; synchronized 3D coach; free**	**Solves all gaps: free web access, automated scanning, and structured learning.**

Table 2: Comparative Analysis of Existing Cubing Solutions vs. Plastic Box Solver

2.3 User Classes and Stakeholder Characteristics

The system serves four primary stakeholder categories:

- **SH-01: First-Time Learners / Casual Owners:** Users who own an unsolved cube but lack knowledge of notation. They require frictionless camera scanning, automatic lighting tolerance, and intuitive step-by-step 3D animations.
- **SH-02: Self-Practice & Intermediate Cubers:** Solvers familiar with basic steps who wish to learn CFOP or improve speed. They require an unassisted interactive timer, scramble generation, and 3D solve replay.
- **SH-03: Competitive Speedcubers:** Advanced practitioners who require keyboard turn shortcuts (U, D, L, R, F, B), rapid queue animation without frame lag, and accurate leaderboard rankings.
- **SH-04: System Administrators & Academic Evaluators:** Instructors assessing project fidelity, test coverage, and code maintainability.

2.4 Operating Environment

The software executes within the following runtime environments:

- **Client-Side Device:** Standard modern web browsers (Chromium 100+, Firefox 110+, Safari 16+) with WebGL 2.0 support and camera access permissions. Fully responsive from 360px mobile screens up to 4K desktops.
- **Backend Server:** Python 3.11+ running FastAPI and Uvicorn on Linux/x86-64 or ARM64 architectures.
- **Database & Deployment:** PostgreSQL 16 managed via SQLAlchemy ORM; containerized using Docker and Docker Compose.

2.5 Design and Implementation Constraints

- **C-01 (Zero-Install Architecture):** The client must run entirely in the browser without third-party browser plugins or native executable downloads.
- **C-02 (Open-Source Ecosystem):** All libraries, frameworks, and tools must carry permissive open-source licenses (MIT, Apache 2.0, BSD).
- **C-03 (Lighting and Hardware Variance):** Camera scanning must account for consumer-grade webcams and fluctuating indoor ambient lighting without requiring dedicated studio setups.
- **C-04 (Academic Schedule):** Development must conclude within the 14-week term of SWE 4404.

2.6 Assumptions and Dependencies

- **Standard Physical Cube:** The physical puzzle is assumed to be a standard $3 \times 3 \times 3$ cube with six solid distinct face colors adhering to the canonical Western color scheme (White opposite Yellow, Green opposite Blue, Red opposite Orange).
- **Browser Hardware Acceleration:** The user's device supports WebGL hardware-accelerated canvas rendering at ≥ 30 FPS.

3 System Features and Functional Requirements

This section details all 24 Functional Requirements (FR-01 through FR-24), organized into six cohesive feature modules corresponding to the system's functional decomposition. Each requirement card is self-contained with unified margin alignment, normative requirement statements, engineering justifications, user stories, and testable acceptance criteria.

3.1 Feature Group 1: User Account and Profile Management

FR-01: New User Registration

IMPLEMENTED MEDIUM PRIORITY

Source: E-01 (Learner Interviews) **Stakeholder:** SH-01, SH-02

Requirement Statement: The system shall allow a new user to register for an account using a unique email address, a unique display name, and a secure password.

Engineering Justification: Cross-session personal best tracking, saved cube slots, and global leaderboard ranking require persistent identity binding. Storing passwords strictly in salted hashed form (`bcrypt`) fulfills mandatory security criteria.

User Story: As a first-time visitor, I want to register with my email, display name, and password, so that I can save my scan history, solve times, and leaderboard ranking across sessions.

Acceptance Criteria:

- Given a visitor provides a valid, unused email, unique display name, and password ($\geq$ 8 chars), when submitted, an account is created and a JWT token is returned.
- Given an email or display name that already exists, the system shall reject submission with an explicit HTTP 400 error.
- Passwords shall be cryptographically salted and hashed using bcrypt before persistence.

FR-02: Log In to Existing Account

IMPLEMENTED MEDIUM PRIORITY

Source: E-01, E-02 **Stakeholder:** SH-01, SH-02, SH-03

Requirement Statement: The system shall authenticate returning users via email and password, issuing a signed JSON Web Token (JWT).

Engineering Justification: Stateless JWT authentication eliminates server-side session overhead, scales seamlessly across Dockerized API instances, and enables persistent authorization via browser local storage.

User Story: As a returning user, I want to log in securely, so that I can resume practicing without losing my personal history or leaderboard standing.

Acceptance Criteria:

- Given valid credentials, the system returns a JWT access token and hydrates client state.
- Given invalid credentials, authentication fails with HTTP 401 and an explanatory message.
- Session tokens persist across page reloads until explicit logout or token expiration.

FR-03: Manage Personal Profile

IMPLEMENTED LOW PRIORITY

Source: E-02 (Speedcuber Testing) **Stakeholder:** SH-02, SH-03

Requirement Statement: The system shall allow authenticated users to view and update their profile display name.

Engineering Justification: Users frequently wish to change their competitive handle without losing past solve records. Re-indexing leaderboard queries by user ID ensures display name changes propagate immediately.

User Story: As a logged-in user, I want to edit my display name, so that my current identity is accurately reflected on the global leaderboard.

Acceptance Criteria:

- Given a new unique display name, saving updates the database and immediately reflects on the dashboard and leaderboard.
- Submissions with taken display names are rejected with clear user feedback.

FR-15: Reset Forgotten Password via Email

DEFERRED MEDIUM PRIORITY

Source: Initial Baseline Specification **Stakeholder:** SH-01, SH-02

Requirement Statement: The system shall allow users to request an automated password reset link dispatched via SMTP email.

Engineering Justification: Deferred during sprint planning. External transactional SMTP services (e.g., SendGrid, Mailgun) introduce external API recurring costs and network configuration dependencies unnecessary for an academic prototype. Replaced by direct profile management.

User Story: As a user who forgot their password, I want an emailed reset link, so that I can regain access without manual administrator intervention.

Acceptance Criteria:

- Documented for future production release; deferred in current laboratory scope.

FR-22: Log Out of Account

IMPLEMENTED LOW PRIORITY

Source: Standard Security Practice **Stakeholder:** SH-01, SH-02, SH-03

Requirement Statement: The system shall allow an authenticated user to terminate their session from any view.

Engineering Justification: Essential for privacy on shared campus computer lab workstations to prevent session hijacking.

User Story: As a user on a shared university computer, I want to log out easily, so that my account remains secure after I leave.

Acceptance Criteria:

- Clicking “Log Out” clears the client JWT token and redirects immediately to the login view.
- Browser back navigation after logout shall not reveal cached protected user data.

3.2 Feature Group 2: Cube Input, Computer Vision, and Validation

FR-04: Scan Physical Cube with Camera IMPLEMENTED HIGH PRIORITY

Source: E-01 (Beginner Pain Points) **Stakeholder:** SH-01

Requirement Statement: The system shall capture all six faces of a physical Rubik's Cube using a connected webcam or smartphone camera via browser `getUserMedia`.

Engineering Justification: Manually entering 54 color tiles is tedious and highly error-prone. A guided 6-step sequential camera flow ($F \rightarrow R \rightarrow B \rightarrow L \rightarrow U \rightarrow D$) with on-screen bounding boxes guides users intuitively.

User Story: As an owner of a scrambled physical cube, I want to scan its faces with my camera, so that I avoid typing 54 sticker colors by hand.

Acceptance Criteria:

- When granted camera permission, the video feed displays a centered 3×3 alignment grid with surrounding orientation edge hints (Top, Bottom, Left, Right).
- The system captures all six faces sequentially and packages images into canonical URFDLB order.
- If camera permission is denied, the system prompts the user and offers the manual 2D net editor.

FR-05: Manually Enter Cube Colors (2D Net) IMPLEMENTED MEDIUM PRIORITY

Source: E-01, E-03 (Fallback Architecture) **Stakeholder:** SH-01, SH-02

Requirement Statement: The system shall provide a clickable 2D cross-net interface for manual color assignment and scan correction.

Engineering Justification: Camera scanning may fail under extreme glare or poor lighting. A clickable 2D net provides a robust fallback and lets users rapidly correct individual misclassified stickers.

User Story: As a user in poor lighting, I want to set sticker colors on a flat 2D cube net, so that I can submit an accurate cube even if the camera scan fails.

Acceptance Criteria:

- Displays a standard unfolded cross net (U, L, F, R, D, B) with a 6-color palette (w, y, r, o, g, b).
- Center stickers are fixed to maintain canonical orientation; clicking unassigned stickers applies the active palette color.
- Keyboard number keys 1–6 provide rapid palette switching shortcuts.

FR-06a: Extract Sticker Colors from Images IMPLEMENTED HIGH PRIORITY

Source: E-01, E-03 **Stakeholder:** SH-01

Requirement Statement: The OpenCV backend shall detect and extract all 54 sticker colors from the six uploaded face frames.

Engineering Justification: Cell boundaries and plastic bevels produce shadow artifacts. Cropping the inner 50% of each cell and computing median BGR values filters specular reflections and surface scratches.

User Story: As a user who scanned their cube, I want the system to extract sticker colors automatically, so that I do not have to manually label 54 cells.

Acceptance Criteria:

- The backend divides each face image into a 3×3 grid and samples the median color of the middle 50% region.
- Produces a canonical 54-character color string mapped to standard URFDLB indexing.

FR-06b: Validate Scanned Cube Legality IMPLEMENTED HIGH PRIORITY

Source: E-01, E-03 **Stakeholder:** SH-01, SH-02

Requirement Statement: The system shall validate that the 54-color configuration represents a physically legal, solvable Rubik's Cube before attempting to solve.

Engineering Justification: A cube with impossible permutations (e.g., duplicated edges, twisted single corners, or incorrect sticker distributions) crashes two-phase solving algorithms. Pre-validation provides actionable, user-friendly guidance.

User Story: As a user who scanned my cube, I want the system to verify that the cube is legal and solvable, so that I immediately know if a face needs rescanning.

Acceptance Criteria:

- Rejects states where any color count does not equal exactly 9.
- Validates edge and corner parity, permutation parity, and total corner/edge orientation sums.
- Returns specific error details identifying which faces or cubies are physically inconsistent.

FR-06c: Confirm Cube State via 2D Preview

IMPLEMENTED HIGH PRIORITY

Source: E-01 **Stakeholder:** SH-01

Requirement Statement: The system shall display a 2D preview of the scanned cube state for user confirmation prior to entering the 3D solve workspace.

Engineering Justification: Serves as a crucial visual checkpoint allowing users to verify computer vision accuracy before initiating automated or guided solving.

User Story: As a user reviewing my scan, I want to see a 2D confirmation preview, so that I can visually verify accuracy before solving begins.

Acceptance Criteria:

- Displays all six faces in an intuitive 2D layout showing the extracted colors.
- Provides direct buttons to "Confirm and Proceed" or "Edit in Net Editor".

FR-17: Auto-Calibrate Color via Centers

IMPLEMENTED HIGH PRIORITY

Source: E-01, E-03 (Lighting Robustness) **Stakeholder:** SH-01

Requirement Statement: The system shall dynamically calibrate color classification for each face by using its center cubelet as the reference anchor.

Engineering Justification: Fixed HSV ranges fail under varying warm/cool room lighting. Because Rubik's Cube center pieces never change spatial relationships, their detected median values provide an adaptive baseline for classifying adjacent stickers via CIE LAB color distance.

User Story: As a user in varying lighting conditions, I want color detection to calibrate automatically from center stickers, so that I do not need manual color adjustments.

Acceptance Criteria:

- Center stickers are extracted first; White is identified by lowest saturation (S).
- The remaining five centers are identified by hue order, and non-center stickers are classified by nearest Euclidean distance in CIE LAB color space.

3.3 Feature Group 3: Interactive 3D Visualization and Manipulation

FR-07: View and Rotate Interactive 3D Cube

IMPLEMENTED HIGH PRIORITY

Source: E-01, E-02 **Stakeholder:** SH-01, SH-02, SH-03

Requirement Statement: The system shall render the cube as an interactive 3D model composed of 26 individual cubie meshes within a WebGL Three.js canvas.

Engineering Justification: A 3D model mirroring the user's physical cube eliminates spatial disorientation. Arcball/Orbit controls allow complete spatial inspection without desynchronizing logical state.

User Story: As a user reviewing my cube, I want to freely orbit and inspect a 3D model mirroring my physical cube, so that I can orient myself before solving.

Acceptance Criteria:

- Renders 26 cubie meshes with rounded bevels and internal dark plastic cores.
- Smooth camera orbiting via mouse drag and touch gestures maintaining $\geq$ 30 FPS.
- Responsive viewport scaling preserving all UI dock elements on screens down to 360px.

FR-18: Undo and Redo Moves in Practice

IMPLEMENTED LOW PRIORITY

Source: E-02 (Self-Practice Flow) **Stakeholder:** SH-02

Requirement Statement: The system shall allow users to undo and redo individual face rotations during Interactive Solve mode.

Engineering Justification: Solvers practicing independently frequently make accidental face turns. Supporting a move history stack avoids forcing users to restart their entire session upon an errant turn.

User Story: As a user practicing my own method, I want to undo an accidental move, so that I can correct my turn without abandoning the attempt.

Acceptance Criteria:

- Clicking "Undo" reverts the 3D model and decrements the move counter.
- Applying a new move after an undo clears the subsequent redo history stack.
- Active solve timing continues uninterrupted during undo/redo actions.

FR-20: Control Cube with Keyboard Shortcuts IMPLEMENTED MEDIUM PRIORITY

Source: E-02 (Speedcuber Demands) **Stakeholder:** SH-02, SH-03

Requirement Statement: The system shall map standard Singmaster notation keys (U, D, L, R, F, B), Shift modifiers (prime turns), and slice turns (M, E, S) to instantaneous 3D layer turns.

Engineering Justification: Advanced speedcubers turn at 4–8 moves per second. Mouse clicks are far too slow; full keyboard shortcuts with queue acceleration prevent input latency.

User Story: As a speedcuber practicing in Interactive Mode, I want to turn faces using notation keys, so that I can execute algorithms as fast as on a physical speedcube.

Acceptance Criteria:

- Pressing R triggers a clockwise turn; Shift+R triggers counter-clockwise (R').
- Arrow keys execute whole-cube spatial rotations (x, y rotations).
- Keypresses in text input fields are isolated and do not trigger cube rotations.

3.4 Feature Group 4: Solving Algorithms and Educational Tutorials

FR-08a: Optimal Solver (Auto-Solve) IMPLEMENTED HIGH PRIORITY

Source: E-03 (Algorithmic Literature) **Stakeholder:** SH-01, SH-02, SH-03

Requirement Statement: The system shall compute an optimal move sequence (≤ 20 moves) using the Kociemba two-phase algorithm within 2 seconds.

Engineering Justification: Kociemba's two-phase algorithm traverses coset spaces ($G_0 \rightarrow G_1 \rightarrow$ Solved), guaranteeing near-God's-number solutions suitable for minimal-move automated demonstrations.

User Story: As a user who wants the quickest solution, I want the system to compute an optimal move list, so that I can see my cube solved with minimal moves.

Acceptance Criteria:

- Returns an optimal move sequence within 2.0 seconds for any legal state.
- Includes total move count and phase decomposition metrics.

FR-08b: Guided Solver (Beginner & CFOP) IMPLEMENTED HIGH PRIORITY

Source: E-01, E-04 (Proposal Panel Advice) **Stakeholder:** SH-01, SH-02

Requirement Statement: The system shall compute human-teachable solution paths using Beginner's Layer-by-Layer (7 stages) and advanced CFOP methods.

Engineering Justification: Optimal mathematical solutions are incomprehensible to human learners because they break and restore intermediate layers simultaneously. Human solvers require progressive substructure milestones (Cross → F2L → OLL → PLL).

User Story: As a user learning to solve, I want a beginner-friendly move list broken into recognizable stages, so that I can understand and memorize the technique.

Acceptance Criteria:

- Segregates moves into discrete stages (White Cross, First Layer Corners, Second Layer Edges, Yellow Cross, Yellow Corners, Permute Corners, Permute Edges).
- Allows selecting between Beginner method and advanced CFOP method.

FR-16: Generate Practice Scramble IMPLEMENTED LOW PRIORITY

Source: E-02 (WCA Standards) **Stakeholder:** SH-02, SH-03

Requirement Statement: The system shall generate randomized, pseudo-random 20-move scramble sequences adhering to World Cube Association (WCA) guidelines.

Engineering Justification: Practice requires repeatable, uniformly distributed random states. Redundant moves (e.g., $R\ R'$ or $U\ U2$) are programmatically filtered out.

User Story: As a user practicing speedcubing, I want to click "New Scramble" to randomize the 3D cube, so that I can start a fresh attempt without manual setup.

Acceptance Criteria:

- Generates ≥ 20 random moves without adjacent identical or cancelling faces.
- Applies the scramble to the 3D model and displays the notation string for optional physical cube replication.

FR-24: Interactive Stage Tutorials

IMPLEMENTED MEDIUM PRIORITY

Source: E-04 (Proposal Panel Advice) **Stakeholder:** SH-01, SH-02

Requirement Statement: The system shall provide dedicated, isolated tutorial practice workspaces for individual solving stages with target formula HUDs.

Engineering Justification: Added post-proposal to address examiner feedback regarding pedagogical depth. Users can isolate and drill difficult algorithms (e.g., Sexy Move, Sune, T-Perm) without having to scramble an entire cube.

User Story: As a beginner struggling with the final layer, I want to practice specific stage formulas on isolated scenarios, so that I can build muscle memory.

Acceptance Criteria:

- Features a Stage Picker catalog covering Daisy, White Cross, F2L, OLL, and PLL.
- Automatically loads stage-specific cube states, displays the exact target algorithm formula, and triggers a celebratory modal upon correct execution.

3.5 Feature Group 5: Solve Execution Modes and Live Coaching

FR-09: Timed Interactive Solve Mode

IMPLEMENTED MEDIUM PRIORITY

Source: E-02 (Speedcuber Testing) **Stakeholder:** SH-02, SH-03

Requirement Statement: The system shall provide an unassisted Interactive Solve mode featuring an automated timer and real-time move counter.

Engineering Justification: Accurate benchmarking requires high-precision timing ($\pm$10ms) that starts on the first player move and automatically halts upon reaching the solved state across any orientation.

User Story: As a cuber practicing my speed, I want a live timer and move counter that runs automatically while I turn the 3D cube, so that I can benchmark my performance.

Acceptance Criteria:

- Timer starts on the first user turn and increments the move counter on every turn.
- Automatically detects when all 6 faces are unified in color (supporting all 24 spatial cube orientations) and freezes the timer immediately.

FR-10: Step-by-Step Guided Coach IMPLEMENTED MEDIUM PRIORITY

Source: E-01 (Beginner Pain Points) **Stakeholder:** SH-01

Requirement Statement: The system shall provide a 7-stage live coaching sidebar that chunks algorithms into human-digestible triggers with step-by-step navigation.

Engineering Justification: Displaying 60 unchunked moves overwhelms beginners. The coaching engine evaluates current cube state predicates, highlights the active substructure, and displays JPerm trigger notation (e.g., [R U R' U']).

User Story: As a beginner, I want to see one move or algorithm trigger at a time in a sidebar and mirror it on my physical cube, so that I can learn without video distraction.

Acceptance Criteria:

- Highlights current move in a dock; "Next" animates the turn on the 3D model, while "Previous" cleanly reverses it.
- Live checklist checks off completed stages dynamically as substructures are solved.

FR-11: Auto-Solve Playback Mode IMPLEMENTED MEDIUM PRIORITY

Source: E-01, E-03 **Stakeholder:** SH-01

Requirement Statement: The system shall animate the full solution sequence automatically with interactive playback controls (Play, Pause, Speed Slider, Step).

Engineering Justification: Passive observation enables users to study the flow of optimal algorithms before attempting physical execution. A dynamic speed slider (0.5× to 2×) accommodates varying cognitive speeds.

User Story: As a curious user, I want to watch the 3D cube solve itself automatically with speed controls, so that I can study the movements at my own pace.

Acceptance Criteria:

- Clicking "Play" initiates continuous automated execution; "Pause" halts on the current frame without move loss.
- Speed slider scales animation duration seamlessly between 100ms and 600ms per turn.

3.6 Feature Group 6: Telemetry, Persistence, Saved Cubes, and Leaderboard

FR-12a: Track Solve Time and Moves

IMPLEMENTED LOW PRIORITY

Source: E-02 **Stakeholder:** SH-02, SH-03

Requirement Statement: The system shall record the exact elapsed duration and total move count upon solve completion.

Engineering Justification: Objective telemetry provides the quantitative empirical basis for evaluating solver skill progression over time.

User Story: As an active solver, I want my time and move count recorded upon completing a solve, so that I have accurate historical records.

Acceptance Criteria:

- Solves store exact floating-point seconds and integer move counts in the database upon user submission.

FR-12b: Solve Efficiency Percentage

IMPLEMENTED LOW PRIORITY

Source: E-02, E-04 (Fairness Refactoring) **Stakeholder:** SH-02, SH-03

Requirement Statement: The system shall compute an efficiency score comparing actual user turns against an algorithmic CFOP baseline: Efficiency = min(100%, (Baseline Moves/Us
100).

Engineering Justification: Originally, efficiency compared user turns against Kociemba (≤ 20 moves), yielding demoralizing 20%–30% ratings for competent human speedcubers. Refactoring to a CFOP baseline (55 moves) provides an equitable, motivating metric.

User Story: As a solver, I want to see my efficiency percentage relative to a realistic human method, so that I know how much room for improvement remains.

Acceptance Criteria:

- Displays efficiency percentage capped at 100% alongside raw time and move count in the solve summary.

FR-13a: Persist Solve History Server-Side

IMPLEMENTED LOW PRIORITY

Source: E-01, E-02 **Stakeholder:** SH-02, SH-03

Requirement Statement: The system shall persist complete solve records (scramble, solution, time, moves, date) associated with the authenticated user in PostgreSQL.

Engineering Justification: Server-side storage prevents data loss from browser cache clearing and enables cross-device access between mobile phones and laptops.

User Story: As a user tracking my journey, I want every completed solve saved to my account on the server, so that my history is accessible from any device.

Acceptance Criteria:

- Dashboard displays a paginated history table (10 solves per page) with dates, times, moves, and efficiency ratings.

FR-13b: Personal Best & 3D Replay

IMPLEMENTED LOW PRIORITY

Source: E-02 (Speedcuber Demands) **Stakeholder:** SH-02, SH-03

Requirement Statement: The system shall identify the user's fastest solve (Personal Best) and allow launching a full step-by-step 3D playback of any historical solve.

Engineering Justification: Competitive cubers need to review their fastest solves to analyze turn choices. Re-playing historical move sequences directly on the 3D engine provides unparalleled diagnostic feedback.

User Story: As a competitive solver, I want to see my Personal Best highlighted and replay it in 3D, so that I can inspect how I achieved my record.

Acceptance Criteria:

- Flags the lowest `solve_time` record (breaking ties by efficiency) on the user dashboard.
- Clicking "Replay" loads the original scramble and solution moves into the 3D workspace for playback.

FR-14: Global Leaderboard Standings

IMPLEMENTED LOW PRIORITY

Source: E-02 **Stakeholder:** SH-02, SH-03

Requirement Statement: The system shall render a global leaderboard ranking all registered users by their fastest verified solve time, breaking ties by efficiency.

Engineering Justification: Social comparison and friendly competition drive recurring user retention and deliberate practice.

User Story: As a competitive cuber, I want to see my rank on a global leaderboard, so that I can measure my standing against peers.

Acceptance Criteria:

- Ranks each user by their single fastest unassisted solve time.
- Visually crowns rank #1 (Gold), #2 (Silver), #3 (Bronze) and highlights the logged-in user's active rank row.

FR-19: Share Solve Result Card

DEFERRED LOW PRIORITY

Source: Initial Baseline Specification **Stakeholder:** SH-02

Requirement Statement: The system shall generate a shareable image card or public URL summarizing solve metrics for external social media export.

Engineering Justification: Deferred during sprint planning to prioritize core pedagogical features (CFOP Live Coach and Stage Tutorials). Users can currently share dashboard stats directly.

User Story: As a solver proud of my time, I want an exportable summary image, so that I can post my achievement on social channels.

Acceptance Criteria:

- Deferred in current release scope.

FR-21: Milestone Badges & Progression

DEFERRED LOW PRIORITY

Source: Initial Baseline Specification **Stakeholder:** SH-01, SH-02

Requirement Statement: The system shall award gamified achievement badges (e.g., "10 Solves", "Sub-60") and retroactively award badges upon adding new thresholds.

Engineering Justification: Gamification badges provide auxiliary motivation but are secondary to functional solving and learning. Deferred to preserve engineering capacity for the 3D queue animation engine and OpenCV pipeline.

User Story: As a user tracking my progression, I want visible milestone badges on my profile, so that I feel rewarded for sustained practice.

Acceptance Criteria:

- Documented in product backlog; deferred in current laboratory scope.

FR-23: Cross-Device Saved Cube Slots

IMPLEMENTED HIGH PRIORITY

Source: E-05 (Industry Expert Feedback) **Stakeholder:** SH-01, SH-02

Requirement Statement: The system shall allow authenticated users to save up to three physical cube states to server-side slots and restore them on any device.

Engineering Justification: Added specifically in response to external industry feedback during the IUT design review. Users frequently scan physical cubes on a smartphone camera in good room lighting, but prefer solving or following tutorials on a large desktop monitor.

User Story: As a user who scanned my cube on my phone, I want to save the cube state to my account, so that I can resume solving it later on my laptop.

Acceptance Criteria:

- Provides full CRUD endpoints (`GET, POST, DELETE /api/cubes`) capped strictly at 3 slots per user account.
- Saved slots appear in the Cube Input view with timestamps, facelet previews, and a one-click "Restore to 3D View" button.

4 External Interface Requirements

4.1 User Interfaces

The user interface follows a modern, dark cyberpunk/neon aesthetic optimized for high contrast and visual clarity. The layout is built with Tailwind CSS and responsive CSS flexbox/grid containers.

- **AppShell Layout:** Standard header with brand iconography, navigation links (*Solve, Input, Tutorials, Dashboard, Leaderboard*), user profile avatar, and a collapsible desktop/mobile navigation drawer.
- **Solve Workspace:** Full-screen 3D canvas with an overlaid collapsible coaching dock, floating move notation chips, playback transport controls, and an orientation

gizmo.

- **Camera Scanner HUD:** Centered translucent guide box with active face indicators, rotational transition prompts (e.g., "Turn cube 90° to left"), and edge color reference swatches.
- **2D Net Editor:** Cross-net unfolding with an interactive 6-color paint palette and sticker reset triggers.

4.2 Hardware Interfaces

- **Digital Video Camera:** Interfaced via the W3C `MediaDevices.getUserMedia` web API. The system captures single-frame JPEG snapshots at a minimum resolution of 640×480 pixels.
- **Physical Keyboard:** Listens for standard QWERTY keydown events mapping notation keys to 3D rotations, with automatic input isolation when focusing HTML form fields.
- **Touchscreen Displays:** Supports single-touch orbital drags and dual-touch pinch-to-zoom gestures, as well as kinematic layer swipes.

4.3 Software Interfaces

- **Relational Database (PostgreSQL 16):** Interfaced via Python SQLAlchemy 2.0 ORM using connection pooling for user, solve, and saved cube tables.
- **Computer Vision Engine (OpenCV 4.x):** Python server-side image processing utilizing `cv2.imdecode`, `cv2.cvtColor` (HSV and CIE LAB conversions), and `cv2.Laplacian` for blur variance.
- **Solving Algorithms:** Native Python implementations of Kociemba's two-phase algorithm and Fridrich CFOP solver.
- **3D Graphics Pipeline:** Three.js and `@react-three/fiber` utilizing client WebGL 2.0 rendering contexts.

4.4 Communications Interfaces

The client communicates with the server strictly via stateless RESTful HTTP/1.1 requests over port 8000 (proxied via Vite during development):

- **Payload Formats:** All API exchanges utilize UTF-8 encoded JSON Data Transfer Objects (DTOs), except camera uploads which use `multipart/form-data`.
- **Security Headers:** Protected endpoints require an `Authorization: Bearer <JWT>` header.
- **Upload Restrictions:** Scan uploads are capped at 5MB per request to prevent Denial-of-Service resource exhaustion.

5 Other Non-Functional Requirements

The non-functional requirements define the quality attributes, performance envelopes, security postures, and operational standards of the system.

ID	Category	Quantitative Target	Engineering Justification
NFR-01	Vision Accuracy	Correct classification under standard 300–800 lux indoor lighting	Ensures casual users can scan cubes in typical residential or campus dorm lighting without specialized photo booths.
NFR-02	Solver Latency	Optimal solve response time < 2.0 seconds	Immediate feedback maintains user engagement; long computational hangs cause user abandonment.
NFR-03	Web Portability	100% zero-install execution on Chromium, Firefox, Safari	Eliminates operating system compatibility friction and removes download barriers for first-time visitors.
NFR-04	3D Frame Budget	≥ 30 FPS (target 60 FPS) during layer animation	Smooth rotational animations prevent motion sickness and visual tearing during rapid manual turns.
NFR-05	State Integrity	100% rejection of physically impossible permutations	Prevents unsolvable or malformed cube inputs from triggering solver infinite loops or backend server exceptions.
NFR-06	Authentication	Salting & bcrypt hashing; JWT expiry ≤ 7 days	Protects stored credentials against rainbow-table attacks and database compromise.
NFR-07	Relational ACID	Zero loss of completed solves or saved cubes across sessions	Guarantees personal bests and history remain immutable once committed to storage.
NFR-08	Responsiveness	Usable on viewports down to 360px width	Accommodates low-end Android mobile devices common among university students.
NFR-09	Error Resilience	User-friendly HTTP 4xx messages; zero unhandled 500 crashes	Clear client error notifications prevent confusion when scans fail due to darkness or blur.

Table 3: Non-Functional Requirements Specification and Justification

6 Traceability, Sustainability, and Project Feasibility

6.1 Requirements Traceability Matrix

Table 4 demonstrates explicit backward and forward traceability from collected stakeholder needs to functional and non-functional requirements.

Req ID	Feature Group	Primary Stakeholder Need	Source	Status
FR-01	Account Management	Secure identity creation	SH-01, SH-02	Implemented
FR-02	Account Management	Persistent session access	SH-01, SH-02	Implemented
FR-03	Account Management	Customizable public handle	SH-02, SH-03	Implemented
FR-04	Cube Input	Automated physical cube capture	SH-01 (Beginners)	Implemented
FR-05	Cube Input	Manual fallback for poor lighting	SH-01, SH-02	Implemented
FR-06a	Vision	Automated sticker color extraction	SH-01	Implemented
FR-06b	Vision / Solver	Legality and solvability check	SH-01, SH-02	Implemented
FR-06c	Vision / Input	Visual verification before solve	SH-01	Implemented
FR-07	3D Visualization	Interactive spatial model	SH-01, SH-02, SH-03	Implemented
FR-08a	Solver	Minimal move auto-solution (≤ 20)	SH-01, SH-03	Implemented
FR-08b	Solver	Human-learnable step-by-step path	SH-01, SH-02	Implemented
FR-09	Solve Mode	Unassisted practice with live timer	SH-02, SH-03	Implemented
FR-10	Solve Mode	7-stage guided coaching dock	SH-01 (Beginners)	Implemented
FR-11	Solve Mode	Passive observation with speed controls	SH-01	Implemented
FR-12a	Telemetry	Quantitative time and move counts	SH-02, SH-03	Implemented
FR-12b	Telemetry	Fair efficiency benchmark (CFOP)	SH-02, SH-03	Implemented
FR-13a	Persistence	Server-side cross-device solve history	SH-02, SH-03	Implemented
FR-13b	Persistence	Personal Best highlighting & 3D replay	SH-02, SH-03	Implemented
FR-14	Leaderboard	Social motivation and global rankings	SH-02, SH-03	Implemented
FR-15	Account Management	Self-service password recovery	SH-01, SH-02	Deferred
FR-16	Solver / Practice	Standard WCA random scrambles	SH-02, SH-03	Implemented
FR-17	Vision	Adaptive ambient color calibration	SH-01	Implemented
FR-18	3D Visualization	Mistake reversal in practice mode	SH-02	Implemented
FR-19	Persistence	Social media result card generation	SH-02	Deferred
FR-20	3D Visualization	High-speed notation keyboard turns	SH-02, SH-03	Implemented
FR-21	Leaderboard	Gamified milestone achievement badges	SH-01, SH-02	Deferred
FR-22	Account Management	Shared workstation session termination	SH-01, SH-02, SH-03	Implemented
FR-23	Persistence	Cross-device 3-slot saved cubes	E-05 (Industry Expert)	Implemented
FR-24	Educational	Stage-specific algorithm drill HUD	E-04 (Proposal Panel)	Implemented
NFR-01–09	Quality Attributes	Usability, security, speed, integrity	All Stakeholders	Implemented

Table 4: Requirements Traceability Matrix (Mapping Needs to Requirements)

6.2 Contextual, Privacy, and Sustainable-Development Considerations

In alignment with the Course Outcomes (CO2 and CO4), the specification explicitly incorporates contemporary societal and sustainability dimensions:

- **E-Waste Reduction (SDG 12: Responsible Consumption):** Commercial smart cubes contain lithium-ion batteries, Bluetooth transceivers, and microcontrollers encased in glued plastic, rendering them un-recyclable electronic waste. Plastic Box Solver operates with any ordinary, existing mechanical cube, extending the lifecycle of millions of existing physical plastic puzzles.
- **Democratization of Spatial STEM Learning (SDG 4: Quality Education):** Solving 3D combinatorial puzzles develops spatial intelligence and algorithmic thinking. Providing a 100% free, browser-accessible tool ensures students from underrepresented or low-income backgrounds can learn without financial barriers.
- **User Privacy and Ephemeral Processing:** Camera frames captured for cube scanning are processed entirely in server memory. No user facial images, webcam photos, or ambient background frames are ever written to disk or persisted in database tables, adhering to strict data minimization principles.
- **Colorblind Inclusivity:** In addition to color swatches, the system displays textual face names (U, D, L, R, F, B) and orientation labels (Top, Bottom, Left, Right) to support color-vision deficient users.

6.3 Effort, Infrastructure, and Cost Feasibility Estimates

Course Outcome CO7 requires reasoned engineering management and economic feasibility estimates:

- **Engineering Effort Estimation:** Over the 14-week lifecycle, the three-person development team committed an estimated total of 420 person-hours (averaging 10 hours/week per member across Frontend, Backend/Solver, and Vision/DevOps).
- **Development Infrastructure Cost ($0):** Built entirely upon open-source software (Node.js, React, FastAPI, OpenCV, PostgreSQL, Docker) and hosted on local university development hardware, incurring zero licensing fees during development.
- **Projected Cloud Deployment Cost ($\leq$ $7/month):** Production deployment requires a single entry-tier VPS (2 vCPU, 2GB RAM on Hetzner or DigitalOcean) hosting the Dockerized stack, or free-tier serverless execution (Vercel frontend + Render backend + Neon PostgreSQL), proving high economic feasibility.
- **Economic Rationale for Deferring FR-15:** Integrating third-party transactional email delivery (e.g., SendGrid) would have introduced recurring monthly subscription fees and domain DNS verification dependencies disproportionate to an academic laboratory project.

A Requirement Collection Document and Live Link

In accordance with the project supervisor's formal instructions, this appendix documents the raw system requirements collection spreadsheet gathered during the initial project elicitation phase.

Live Google Sheets Document Link:
https://docs.google.com/spreadsheets/d/1kGwzly7FVnI9GJHZ7Gggss_BEgM_Vx–_m-IrGDG7YA/

Note on Document History: The spreadsheet above represents the initial unedited collection artifact. Over the course of the project, user stories were implemented and refined as detailed in Section 3 and Table 4.

The complete 6-page requirement collection document is embedded below:

Group ID	6
Student ID	230042104, 230042110, 230042118
Title	Plastic Box Solver
Brief Description	A browser-based Rubik's Cube scanning and solving assistant. Users scan or manually input their physical cube's state, the system validates and renders it as an interactive 3D model, then computes a solution (optimal via the Kociemba two-phase algorithm, or beginner's-method for guided learning). Users can solve interactively with a live timer, follow guided step-by-step instructions, or watch an automatic solve animation, with results (time, move count, efficiency) tracked, saved to history, and ranked on a global leaderboard.

---	FR ID	Functional Requirement	User Story ID	US Title	As a (precise role)	I want (requirement)	So that (completion condition)	User Story	Acceptance Criteria	Priority	Status				
1	FR-01	The system shall allow a new user to register for an account using email and password.	US-01	New User Registration	first-time visitor to the platform	to create an account using my email address and a password	I can save my scan history, solve times, and leaderboard rank across sessions	As a first-time visitor to the platform, I want to create an account using my email address and a password, so that I can save my scan history, solve times, and leaderboard rank across sessions.	• Given a visitor enters a valid, unused email and a password of at least 8 characters, when they submit the registration form, then an account is created and the user is redirected to the Main Menu. • Given a visitor enters an email that already exists in the system, when they submit the form, then the system displays 'Email already registered' and does not create a duplicate account. • Given a visitor enters a password shorter than 8 characters, when they submit the form, then the system blocks submission and shows an inline validation error. • Given a registration succeeds, when the password is persisted, then it is stored only in hashed form, never in plaintext.	Medium	To Do				
2	FR-02	The system shall allow a registered user to log in using their credentials.	US-02	Log In to Existing Account	registered user returning to the app	to log in with my email and password	I can resume practicing without losing my saved history or leaderboard standing	As a registered user returning to the app, I want to log in with my email and password, so that I can resume practicing without losing my saved history or leaderboard standing.	• Given a registered user enters correct credentials, when they submit the login form, then they are authenticated and taken to the Main Menu. • Given a user is successfully authenticated, when the session is created, then it persists across page reloads until logout or expiry. • Given an unauthenticated user tries to access a protected page directly by URL, when the page loads, then they are redirected to the login screen.	Medium	To Do				
3	FR-03	The system shall allow a logged-in user to view and update their personal profile.	US-03	Manage Personal Profile	logged-in user	to view and edit my profile details such as display name and avatar	my identity is shown correctly on my solve history and the leaderboard	As a logged-in user, I want to view and edit my profile details such as display name and avatar, so that my identity is shown correctly on my solve history and the leaderboard.	• Given a user opens the profile page, when it loads, then their current display name and email are shown. • Given a user changes their display name to a valid, unique value, when they save, then the new name is reflected immediately on the leaderboard. • Given a user attempts to set a display name already taken by another user, when they save, then the system rejects the change with an explanatory error. • Given a user updates their profile, when the update completes, then a success confirmation is shown.	Low	To Do				
4	FR-04	The system shall accept a scrambled cube state captured via the device camera.	US-04	Scan Physical Cube with Camera	user who owns a physical Rubik's Cube	to scan each of the six faces using my phone or webcam camera	I don't have to manually enter every sticker color by hand	As a user who owns a physical Rubik's Cube, I want to scan each of the six faces using my phone or webcam camera, so that I don't have to manually enter every sticker color by hand.	• Given the user grants camera permission, when they align a cube face inside the on-screen guide frame, then the system captures the image for that face. • Given all six faces have been captured, when the last face is submitted, then the backend receives all six images for color processing. • Given camera permission is denied, when the user opens the scan screen, then the system shows an explanatory message and offers the manual input option. • Given a captured image is blurry or too dark to process, when it is submitted, then the system prompts the user to retake that specific face.	High	In Progress				
5	FR-05	The system shall provide a manual click-input interface for setting sticker colors when camera scanning is unreliable.	US-05	Manually Enter Cube Colors	user in poor lighting conditions	to set each of the 54 sticker colors by clicking on a flat 2D cube net	I can still submit an accurate cube state when camera scanning fails	As a user in poor lighting conditions, I want to set each of the 54 sticker colors by clicking on a flat 2D cube net, so that I can still submit an accurate cube state when camera scanning fails.	• Given the manual input screen is open, when the user clicks a sticker cell, then a color palette appears for selecting that sticker's color. • Given the user has not yet assigned a color to every one of the 54 stickers, when they attempt to submit, then the system blocks submission and highlights unfilled stickers. • Given all 54 stickers are assigned, when the user submits, then the cube state is sent for validation using the same rules as camera-scanned input. • Given the user wants to start over, when they click 'Reset', then all 54 stickers are cleared back to an unset state.	Medium	To Do				
6	FR-06a	The system shall detect and extract all 54 sticker colors from the six submitted face images.	US-06a	Extract Sticker Colors from Scanned Images	user who just finished scanning their cube	the system to automatically extract all 54 sticker colors from my six submitted face images	I don't have to manually identify each sticker color myself	As a user who just finished scanning their cube, I want the system to automatically extract all 54 sticker colors from my six submitted face images, so that I don't have to manually identify each sticker color myself.	• Given six submitted face images, when OpenCV processes them, then exactly nine colors per face are extracted and mapped to the 54 sticker positions.	High	In Progress				

---	FR ID	Functional Requirement	User Story ID	US Title	As a (precise role)	I want (requirement)	So that (completion condition)	User Story	Acceptance Criteria	Priority	Status
7	FR-06b	The system shall validate that the extracted 54-color cube state is legal and solvable, rejecting invalid configurations with specific feedback.	US-06b	Validate Scanned Cube State Legality	user who just finished scanning their cube	the system to check that my scanned colors form a legal, solvable cube	I find out immediately if I need to rescan a face instead of getting a failed solve later	As a user who just finished scanning their cube, I want the system to check that my scanned colors form a legal, solvable cube, so that I find out immediately if I need to rescan a face instead of getting a failed solve later.	• Given the extracted 54-color state has an illegal color distribution (e.g. more than 9 stickers of one color), when validation runs, then the system rejects the state and tells the user which face(s) to rescan. • Given the extracted state has a legal color count but an unsolvable permutation or orientation, when validation runs, then the system rejects it with a message explaining the cube appears physically inconsistent.	High	In Progress
8	FR-06c	The system shall display a 2D preview of the validated cube state for user confirmation before solving begins.	US-06c	Confirm Cube State via 2D Preview	user whose scanned cube state has passed validation	to see a 2D preview of my confirmed cube state	I can visually verify the scan is accurate before proceeding to solve	As a user whose scanned cube state has passed validation, I want to see a 2D preview of my confirmed cube state, so that I can visually verify the scan is accurate before proceeding to solve.	• Given the extracted 54-color state passes all legality checks, when validation completes, then the user is shown a 2D preview for final confirmation before solving begins.	High	In Progress
9	FR-07	The system shall render the confirmed cube state as a fully interactive, rotatable 3D model in the browser.	US-07	View and Rotate Interactive 3D Cube	user reviewing their scanned cube	to freely rotate and inspect a 3D model that mirrors my physical cube exactly	I can verify the scan is correct and get oriented before I start solving	As a user reviewing their scanned cube, I want to freely rotate and inspect a 3D model that mirrors my physical cube exactly, so that I can verify the scan is correct and get oriented before I start solving.	• Given a validated cube state, when the 3D view loads, then all 26 cubies render with colors matching the confirmed 2D preview. • Given the user drags on the 3D view, when they release, then the camera orbits smoothly around the cube without lag on both desktop and mobile Chromium browsers. • Given the user triggers a face rotation via an on-screen button or swipe or keyboard input, when the animation plays, then the visual state and the underlying logical cube state stay in sync. • Given the 3D view is displayed on a screen as narrow as 360px, when it renders, then all controls remain usable without overlapping elements.	High	In Progress
10	FR-08a	The system shall compute an optimal solution using the Kociemba two-phase algorithm for Auto-Solve mode.	US-08a	Generate Optimal Solving Algorithm (Auto-Solve)	user who has confirmed their scanned cube state and selected Auto-Solve	the system to calculate an optimal solving sequence using the Kociemba two-phase algorithm	I can watch my exact cube get solved automatically with minimal moves	As a user who has confirmed their scanned cube state and selected Auto-Solve, I want the system to calculate an optimal solving sequence using the Kociemba two-phase algorithm, so that I can watch my exact cube get solved automatically with minimal moves.	• Given a validated, solvable cube state, when the user requests Auto-Solve, then the Kociemba two-phase algorithm returns a move sequence within 2 seconds. • Given the solver receives an unsolvable or malformed state, when it attempts to compute, then it returns a clear error instead of crashing or hanging. • Given a solution is computed, when it is returned to the frontend, then it includes the total move count for later efficiency comparison.	High	In Progress
11	FR-08b	The system shall compute a beginner's-method solution for Guided Mode.	US-08b	Generate Guided Solving Algorithm	user who has confirmed their scanned cube state and selected Guided Mode	the system to calculate a beginner's-method move list for my exact cube	I can follow it as a step-by-step guide instead of an optimal solution	As a user who has confirmed their scanned cube state and selected Guided Mode, I want the system to calculate a beginner's-method move list for my exact cube, so that I can follow it as a step-by-step guide instead of an optimal solution.	• Given the user selects Guided Mode, when a solution is requested, then the system returns a beginner's-algorithm-based move list instead of a raw optimal solution. • Given the solver receives an unsolvable or malformed state, when it attempts to compute, then it returns a clear error instead of crashing or hanging. • Given a solution is computed, when it is returned to the frontend, then it includes the total move count for later efficiency comparison.	High	In Progress
12	FR-09	The system shall support an Interactive Solve mode where the user manipulates the 3D cube themselves.	US-09	Solve the Cube Myself with Live Timer	user practicing their own solving method	to make my own moves on the 3D cube while a timer and move counter run live	I can measure my solving speed and efficiency without any hints	As a user practicing their own solving method, I want to make my own moves on the 3D cube while a timer and move counter run live, so that I can measure my solving speed and efficiency without any hints.	• Given the user enters Interactive Mode, when they make the first move, then the timer starts automatically and the move counter increments with every subsequent move. • Given the user reaches a fully solved state, when the last move is made, then the timer stops immediately and the final time and move count are recorded. • Given a solve is completed, when results are displayed, then the optimal move count from FR-08a is shown alongside the user's move count for comparison. • Given the user wants to abandon the attempt, when they click 'Reset Cube', then the timer, move counter, and cube state all reset to the original scrambled state.	Medium	To Do

---	FR ID	Functional Requirement	User Story ID	US Title	As a (precise role)	I want (requirement)	So that (completion condition)	User Story	Acceptance Criteria	Priority	Status				
13	FR-10	The system shall support a Guided Solve mode that presents move-by-move instructions following traditional beginner's algorithm.	US-10	Follow Step-by-Step Guided Instructions	user learning to solve the cube for the first time	to see one move at a time in a sidebar and apply it on my physical cube before advancing	I can learn beginner's algorithms without switching attention between a tutorial video and my cube	As a user learning to solve the cube for the first time, I want to see one move at a time in a sidebar and apply it on my physical cube before advancing, so that I can learn beginner's algorithms without switching attention between a tutorial video and my cube.	• Given Guided Mode is active, when the beginner's-algorithm move list loads, then the current move is visually highlighted in the sidebar and mirrored on the 3D model. • Given the user clicks 'Next', when the current move is applied, then the 3D cube animates that single move and advances the highlight to the following move. • Given the user clicks 'Previous', when triggered, then the last applied move is undone on the 3D model and the highlight moves back one step. • Given the user reaches the final move in the list, when it is applied, then the system confirms the cube is fully solved and offers to return to the Main Menu.	Medium	To Do				
14	FR-11	The system shall support an Auto-Solve mode that animates the full computed solution sequence without user input.	US-11	Watch the Cube Solve Itself Automatically	user who wants to see the solution demonstrated	to trigger Auto-Solve and watch the entire move sequence animate on its own	I can study the solving technique passively before attempting it myself	As a user who wants to see the solution demonstrated, I want to trigger Auto-Solve and watch the entire move sequence animate on its own, so that I can study the solving technique passively before attempting it myself.	• Given the user selects Auto-Solve, when they click 'Play', then the full move sequence animates automatically at a consistent, smooth frame rate. • Given the animation is playing, when the user clicks 'Pause', then the animation halts on the current move and can be resumed from the same point. • Given the user wants to review a specific moment, when they drag the progress scrubber, then the 3D cube jumps to the corresponding move in the sequence. • Given the animation reaches the last move, when it completes, then the cube visually matches a fully solved state and playback controls reset. • Given the user is watching an auto solve, when they adjust the speed (slider/button), then the animation speed changes accordingly without requiring them to click next.	Medium	To Do				
15	FR-12a	The system shall track solve time and move count per session in Interactive Mode.	US-12a	Track Solve Time and Move Count	user practicing in Interactive Mode	my solve time and move count to be tracked automatically during my attempt	I have accurate raw data recorded for every attempt without manual timing	As a user practicing in Interactive Mode, I want my solve time and move count to be tracked automatically during my attempt, so that I have accurate raw data recorded for every attempt without manual timing.	• Given a completed solve, when results are calculated, then the recorded time and move count are stored for that session. • Given results are shown, when the user clicks 'Save', then the session record is persisted to their solve history.	Low	To Do				
16	FR-12b	The system shall calculate an efficiency percentage comparing the user's move count to the system-generated solution.	US-12b	View Solve Efficiency Percentage	user who just completed a solve in Interactive Mode	to see my efficiency percentage compared to the system-generated solution	I know exactly how much room for improvement I have	As a user who just completed a solve in Interactive Mode, I want to see my efficiency percentage compared to the system-generated solution, so that I know exactly how much room for improvement I have.	• Given a completed solve, when results are calculated, then efficiency is computed as system-generated move count divided by the user's move count, expressed as a percentage. • Given the calculated efficiency, when it is displayed, then it is shown alongside raw solve time and move count in the results panel. • Given the user's move count is lower than or equal to the system-generated count, when efficiency is computed, then it is capped at 100% rather than exceeding it.	Low	To Do				
17	FR-13a	The system shall persist user solve history server-side.	US-13a	Persist Solve History	user tracking long-term progress	every completed solve to be saved to my account on the server	my history is available no matter which device I log in from	As a user tracking long-term progress, I want every completed solve to be saved to my account on the server, so that my history is available no matter which device I log in from.	• Given a user has completed at least one saved solve, when they open the History page, then every past solve is listed with date, time, move count, and efficiency. • Given a user has solved on both desktop and mobile, when they log in from either device, then their complete history appears identically since it is stored server-side. • Given a user has no completed solves yet, when they open the History page, then the system shows an empty-state message instead of an error.	Low	To Do				
18	FR-13b	The system shall identify and display the user's personal best solve from their history.	US-13b	View Personal Best	user tracking long-term progress	my fastest recorded solve to be flagged as my Personal Best	I can quickly see the benchmark I'm trying to beat	As a user tracking long-term progress, I want my fastest recorded solve to be flagged as my Personal Best, so that I can quickly see the benchmark I'm trying to beat.	• Given a user's history contains multiple solves, when the page loads, then the single fastest solve is flagged as the user's Personal Best.	Low	To Do				

---	FR ID	Functional Requirement	User Story ID	US Title	As a (precise role)	I want (requirement)	So that (completion condition)	User Story	Acceptance Criteria	Priority	Status
19	FR-14	The system shall display a global leaderboard ranking users by solve performance.	US-14	Compare My Rank on the Global Leaderboard	competitive user comparing progress against others	to view a global leaderboard ranked by solve time and efficiency, with my own rank highlighted	I stay motivated to practice and can see how I measure up against other users	As a competitive user comparing progress against others, I want to view a global leaderboard ranked by solve time and efficiency, with my own rank highlighted, so that I stay motivated to practice and can see how I measure up against other users.	• Given multiple users have saved solves, when the leaderboard loads, then entries are sorted by best solve time by default, with ties broken by higher efficiency. • Given the logged-in user appears anywhere on the leaderboard, when it renders, then their own row is visually highlighted even if they must scroll to see it. • Given a new personal best is saved, when the leaderboard is next viewed, then it reflects the updated ranking without requiring more than one manual page refresh. • Given the leaderboard has more entries than fit on one screen, when the user scrolls or paginates, then additional ranked entries load correctly in order.	Low	To Do
20	FR-15	The system shall allow a registered user to reset a forgotten password via an emailed reset link.	US-15	Reset Forgotten Password	registered user who has forgotten their password	to request a password reset link sent to my registered email	I can regain access to my account without contacting support	As a registered user who has forgotten their password, I want to request a password reset link sent to my registered email, so that I can regain access to my account without contacting support.	• Given a user enters a registered email on the 'Forgot Password' screen, when they submit it, then a time-limited reset link is emailed to that address. • Given a user enters an email not associated with any account, when they submit it, then the system shows a generic confirmation message without revealing whether the account exists. • Given a user clicks a valid, unexpired reset link, when they submit a new	Medium	To Do
21	FR-16	The system shall generate a randomized, solvable scramble sequence for practice sessions.	US-16	Generate Practice Scramble	user who wants to practice without a physical mixed-up cube	to click 'New Scramble' and have the 3D cube randomize into a fresh, solvable state	I can start a new practice attempt immediately without manually mixing my physical cube	As a user who wants to practice without a physical mixed-up cube, I want to click 'New Scramble' and have the 3D cube randomize into a fresh, solvable state, so that I can start a new practice attempt immediately without manually mixing my physical cube.	• Given the user clicks 'New Scramble', when the request is processed, then the system generates a sequence of at least 20 random face turns and applies it to the 3D cube. • Given a scramble is generated, when it is applied, then the resulting cube state is guaranteed solvable, matching the same legality rules used for camera-scanned states. • Given a new scramble is requested, when it is generated, then the exact	Low	To Do
22	FR-17	The system shall automatically calibrate color detection for each face by reading the center cubelet as the reference color.	US-17	Auto-Calibrate Color via Center Cubelet	user scanning a cube under varying lighting conditions	the system to automatically determine the base color of each face from its center cubelet	I don't have to manually calibrate colors or worry about lighting differences across faces	As a user scanning a cube under varying lighting conditions, I want the system to automatically determine the base color of each face from its center cubelet, so that I don't have to manually calibrate colors or worry about lighting differences across faces.	• Given a user scans a face, when the image is processed, then the system identifies the color of the center cubelet and uses it as the reference for that face's color. • Given the center cubelet color is determined, when the other 8 cubelets are evaluated, then their colors are classified relative to the 6 reference center colors. • Given the system relies on center cubelets, when no manual calibration	High	In Progress
23	FR-18	The system shall allow a user to undo and redo individual moves during an Interactive Mode solve.	US-18	Undo and Redo Moves During Interactive Solve	user practicing their own solving method in Interactive Mode	to undo my last move or redo a move I just undid	I can correct an accidental turn without resetting my entire attempt	As a user practicing their own solving method in Interactive Mode, I want to undo my last move or redo a move I just undid, so that I can correct an accidental turn without resetting my entire attempt.	• Given at least one move has been made in the current attempt, when the user clicks 'Undo', then the 3D cube reverts to the state before that move and the move counter decrements by one. • Given a move has just been undone, when the user clicks 'Redo', then the undone move is reapplied and the move counter increments back. • Given the user makes a brand-new move after undoing one or more	Low	To Do
24	FR-19	The system shall let a user export or share a summary of a completed solve, including time, move count, and efficiency.	US-19	Share Solve Result	user who just completed a solve they are proud of	to generate a shareable summary card of my time, move count, and efficiency	I can post or send my result outside the platform without manually re-typing the numbers	As a user who just completed a solve they are proud of, I want to generate a shareable summary card of my time, move count, and efficiency, so that I can post or send my result outside the platform without manually re-typing the numbers.	• Given a completed and automatically saved solve, when the user clicks 'Share Result', then the system generates an image or link summarizing time, move count, efficiency, and solve mode. • Given a share link is generated, when it is opened by anyone, then it displays a read-only summary without exposing the viewer's own account data. • Given the user clicks 'Copy Link' or 'Download Image', when the action	Low	To Do
25	FR-20	The system shall allow a user to perform cube face rotations using keyboard shortcuts, mapping standard cube notation keys (U, D, L, R, F, B) to clockwise turns and their Shift-modified versions to counter-clockwise turns.	US-20	Control Cube with Keyboard Shortcuts	user practicing their own solving method in Interactive Mode	to press notation keys like R, U, L, D, F, B (and Shift+key for the reverse direction) to rotate cube faces instead of clicking	I can turn the cube as fast as I would with a physical speedcube, without breaking my flow to reach for a mouse	As a user practicing their own solving method in Interactive Mode, I want to press notation keys like R, U, L, D, F, B (and Shift+key for the reverse direction) to rotate cube faces instead of clicking, so that I can turn the cube as fast as I would with a physical speedcube, without breaking my flow to reach for a mouse.	• Given Interactive Mode is active and the cube has focus, when the user presses R, then the right face rotates 90° clockwise and the move is logged in the move counter as "R". • Given Interactive Mode is active, when the user presses Shift+R, then the right face rotates 90° counter-clockwise and the move is logged as "R'". • Given the user presses any of U, D, L, R, F, B (with or without Shift), when the	Medium	To Do
26	FR-21a	The system shall detect when a user crosses a predefined milestone threshold (cumulative solve count or individual solve time) and award the corresponding badge.	US-21a	Earn Milestone Achievement Badge	returning user tracking long-term progress	to automatically receive a badge when I hit a milestone, such as completing my 10th solve or finishing a solve in under 100 seconds	I stay motivated by visible markers of progress instead of only comparing raw numbers	As a returning user tracking long-term progress, I want to automatically receive a badge when I hit a milestone, such as completing my 10th solve or finishing a solve in under 100 seconds, so that I stay motivated by visible markers of progress instead of only comparing raw numbers.	• Given a user completes and saves their 10th total solve, when the save is processed, then the "10 Solves" badge is awarded to their account and a congratulatory notification is shown. • Given a user completes and saves a solve with a recorded time under 100 seconds, when the save is processed, then the "Sub-100" badge is awarded if the user does not already have it. • Given a user has already earned a specific badge, when they complete	Low	To Do

---	FR ID	Functional Requirement	User Story ID	US Title	As a (precise role)	I want (requirement)	So that (completion condition)	User Story	Acceptance Criteria	Priority	Status				
27	FR-21b	The system shall display all badges earned by a user on their profile, ordered by date unlocked.	US-21b	View Earned Badges on Profile	user who has earned one or more badges	to see all my earned badges on my profile, in the order I unlocked them	I can visibly track my progression over time	As a user who has earned one or more badges, I want to see all my earned badges on my profile, in the order I unlocked them, so that I can visibly track my progression over time.	• Given a user views their profile, when the page loads, then all earned badges are displayed in the order they were unlocked, each with the date earned.	Low	To Do				
28	FR-21c	The system shall retroactively evaluate existing users' historical solves when an admin adds a new milestone threshold.	US-21c	Retroactive Badge Evaluation on New Threshold	admin adding a new milestone threshold	existing users' historical solves to be automatically re-evaluated against the new threshold	already-qualifying users receive the new badge without needing to complete a new solve	As an admin adding a new milestone threshold, I want existing users' historical solves to be automatically re-evaluated against the new threshold, so that already-qualifying users receive the new badge without needing to complete a new solve.	• Given the system defines milestone thresholds (e.g. 10, 50, 100, 500 solves; sub-100s, sub-60s, sub-30s times), when a new threshold is added by an admin, then existing users' historical solves are retroactively evaluated so already-qualifying users receive the new badge without needing a new solve.	Low	To Do				
29	FR-22	The system shall allow a user to log out of their account from any page.	US-22	Log Out of Account	logged-in user	to log out of my account	I can end my session and protect my account on shared devices	As a logged-in user, I want to log out of my account, so that I can end my session and protect my account on shared devices.	• Given a logged-in user clicks 'Log Out', when the action is confirmed, then their session is terminated and they are redirected to the login screen. • Given a user has logged out, when they try to access a protected page using the browser back button, then they are redirected to the login screen instead of seeing cached content. • Given a user is mid-solve in Interactive Mode when they log out, when they log back in, then their in-	Low	To Do				

B Technical Glossary and Acronyms

AUF (Adjust Upper Face): A final U, U', or $U2$ turn made to align the top layer with the rest of the cube after completing the last stage of CFOP.

CIE LAB: A color space defined by the International Commission on Illumination expressed as three values: L^* (perceptual lightness), a^* (green-red axis), and b^* (blue-yellow axis). Used for Euclidean color distance measurement.

CFOP Method: The standard speedcubing method developed by Jessica Fridrich, comprising four stages: Cross, First Two Layers (F2L), Orientation of the Last Layer (OLL), and Permutation of the Last Layer (PLL).

Cubie: One of the 26 physical smaller blocks (corner, edge, or center piece) that assemble into a Rubik's Cube.

DTO (Data Transfer Object): A structured data schema (Pydantic model) utilized to transfer data over HTTP between client and server.

Facelet: One of the 54 individual colored square stickers visible on the outer faces of a $3 \times 3 \times 3$ cube.

HSV: Hue, Saturation, and Value; an alternative cylindrical coordinate representation of RGB color space widely used in computer vision color thresholding.

Kociemba Two-Phase Algorithm: A sub-optimal solving algorithm created by Herbert Kociemba that reduces a cube to the $G_1 = \langle U, D, R2, L2, F2, B2 \rangle$ subgroup in Phase 1, and solves the subgroup using only half-turns in Phase 2, finding solutions in ≤ 20 moves.

PB (Personal Best): The single fastest recorded solve time achieved by an individual speedcuber.

Singmaster Notation: The international standard notation (U, D, L, R, F, B) developed by David Singmaster for recording Rubik's Cube moves, where an apostrophe $(')$ indicates a counter-clockwise $90°$ turn and 2 indicates a $180°$ turn.

URFDLB Order: The canonical face ordering convention: Up, Right, Front, Down, Left, Back.

WCA (World Cube Association): The worldwide governing body that regulates Rubik's Cube competitions and standardizes official scramble generation.